COSC2111/2110 Data Mining
Classification

Xiaodong Li

Computer Science, RMIT University, Melbourne, Australia

Semester 2, 2021

How good is a classifier? Train and test split

- We need to measure its accuracy on known data that WAS NOT used in its construction.
- Randomly divide all available data into two files: TRAINING and TEST. Use the TRAINING file to build the classifier, use the TEST file to measure its accuracy.
- Essentially statistical estimation: We take a random sample from the population of interest and compute a parameter.
- Heuristic: Use 2/3 for training and 1/3 for testing
- Use this method if there are more than 1,000 examples
- Technically speaking this is an estimate of the TRUE ERROR RATE, i.e., errors the deployed classifier will make

Further reading: Section 5.1 of Witten, et al. (2011).

How good is a classifier? Cross validation

- n-fold cross validation.
  Randomly split all available data into n files (folds)
  Repeat n times:
  Learn a classifier using $n - 1$ folds as training data
  Measure error on the fold left out
  Error rate is the average of the n errors
- Ten (10) is a commonly used number of folds and is the WEKA default
- Some people do the 10 fold procedure 10 times
- Stratified: The split between training and test preserves class frequency
- Use for up to 1,000 examples
- For less than 50 examples use leave-one-out

Further reading: Section 5.3 of Witten, et al. (2011). See also:
`https://en.wikipedia.org/wiki/Cross-validation_(statistics)`

Over-fitting

- Also called over-training
- A major problem in data mining/machine learning
- Classifier does not generalize well
- The classifier works well on the data from which it was derived but not on other data from the same population
- High accuracy on training data, low accuracy on test data

See also: `https://en.wikipedia.org/wiki/Overfitting`

The ZeroR Classifier

- Don't look at the attributes
- Find the most common class
- Assign this class to all unknown examples
- If a classifier can't beat ZeroR it hasn't learnt anything

What is wrong with this procedure?

- Take the first 100 examples in the `iris.arff` as `train.arff`
- Take the last 50 examples in the `iris.arff` as `test.arff`
- Run weka, and put in `train.arff` as training file and `test.arff` as the test file

Test/train outcomes

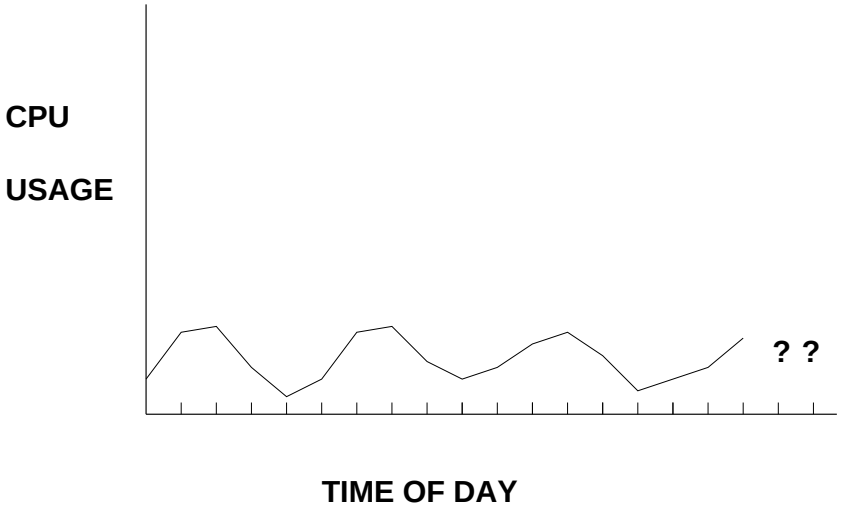
- What would you conclude from the following outcomes for a two class problem:
  - Train accuracy 99%, Test accuracy 80%
  - Train accuracy 75% Test accuracy 73%
  - Train accuracy 53% Test accuracy 52%

Numerical Prediction 1

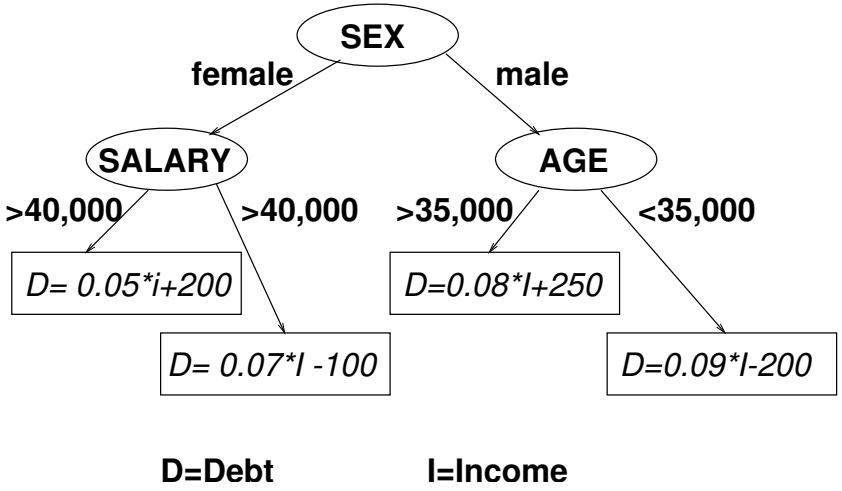
- The class variable is numeric
  - Option 1: Discretise the class variable and use a standard classifier
    - Example: Age is numeric between 0 and 130
    - Baby = 0,2; Preschool = 3-5; Primary = 6-12; etc
    - Weka “[˜ ,2]”; “[3,5]”; “[6-12]”; etc
    - There are a dozen methods
  - Option 2: Use a classifier specifically designed for numeric prediction.
- Note, some classifiers only accept symbolic attribute values, and numeric values need to be discretised.

Numerical Prediction 2

- Time series forecasting



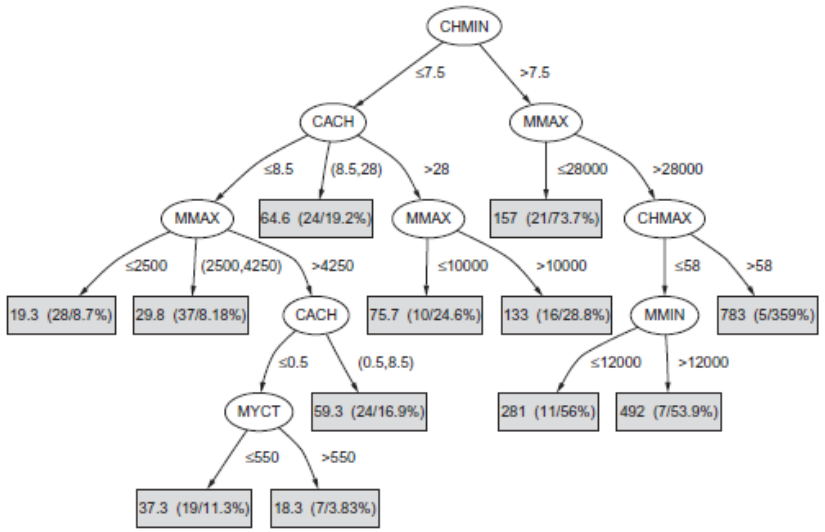
- Regression/model tree

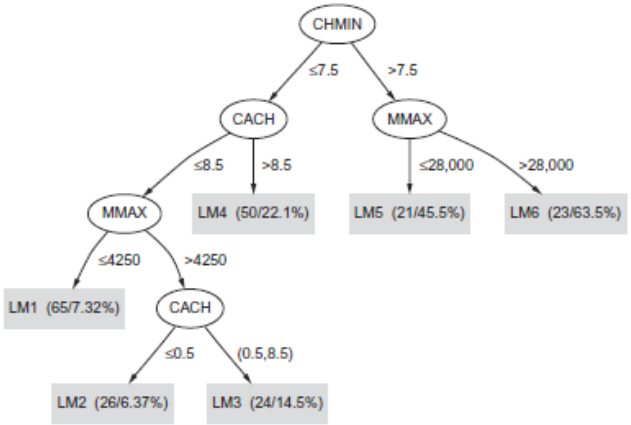


- Root mean squared error

Numerical Prediction 2

PRP =
– 56.1
+ 0.049 MYCT
+ 0.015 MMIN
+ 0.006 MMAX
+ 0.630 CACH
– 0.270 CHMIN
+ 1.46 CHMAX





LM1 PRP = 8.29 + 0.004 MMAX + 2.77 CHMIN
LM2 PRP = 20.3 + 0.004 MMIN – 3.99 CHMIN
+ 0.946 CHMAX
LM3 PRP = 38.1 + 0.012 MMIN
LM4 PRP = 19.5 + 0.002 MMAX + 0.698 CACH
+ 0.969 CHMAX
LM5 PRP = 285 1.46 MYCT + 1.02 CACH
– 9.39 CHMIN
LM6 PRP = – 65.8 + 0.03 MMIN – 2.94 CHMIN
+ 4.98 CHMAX

(A) (B) (C)

FIGURE 3.4

Models for the CPU performance data: (A) linear regression; (B) regression tree; (C) model tree.

See Section 3.4 of W&F.

Accuracy of Numeric Prediction

- Mean Absolute Error

$$\frac{\sum_{i=1}^{n} |actual(i) - predicted(i)|}{n}$$

- Root Mean Squared Error

$$\sqrt{\sum_{i=1}^{n} (actual(i) - predicted(i))^2 / n}$$

where n is the number of examples in the test set

- Why not just add up the errors?

Performance Measures for Numeric Prediction

Table 5.8 Performance Measures for Numeric Prediction

Mean-squared error	$\frac{(p_1-a_1)^2+\cdots+(p_n-a_n)^2}{n}$
Root mean-squared error	$\sqrt{\frac{(p_1-a_1)^2+\cdots+(p_n-a_n)^2}{n}}$
Mean absolute error	$\frac{\lvert p_1-a_1\rvert+\cdots+\lvert p_n-a_n\rvert}{n}$
Relative squared error	$\frac{(p_1-a_1)^2+\cdots+(p_n-a_n)^2}{(a_1-\bar{a})^2+\cdots+(a_n-\bar{a})^2}$ (in this formula and the following two, $\bar{a}$ is the mean value over the training data)
Root relative squared error	$\sqrt{\frac{(p_1-a_1)^2+\cdots+(p_n-a_n)^2}{(a_1-\bar{a})^2+\cdots+(a_n-\bar{a})^2}}$
Relative absolute error	$\frac{\lvert p_1-a_1\rvert+\cdots+\lvert p_n-a_n\rvert}{\lvert a_1-\bar{a}\rvert+\cdots+\lvert a_n-\bar{a}\rvert}$
Correlation coefficient	$\frac{S_{PA}}{\sqrt{S_P S_A}}$, where $S_{PA}=\frac{\sum_i(p_i-\bar{p})(a_i-\bar{a})}{n-1}$, $S_P=\frac{\sum_i(p_i-\bar{p})^2}{n-1}$, $S_A=\frac{\sum_i(a_i-\bar{a})^2}{n-1}$ (here, $\bar{a}$ is the mean value over the test data)

Numeric Prediction Example

Computer performance on benchmarks

Predict CPU time from other variables

```
%
% As used by Kilpatrick, D. & Cameron-Jones, M. (1998). Numeric prediction
% using instance-based learning with encoding length selection. In Progress
% in Connectionist-Based Information Systems. Singapore: Springer-Verlag.
%

@relation 'cpu'
@attribute vendor { adviser, amdahl, apollo, basf, bti, burroughs, c.r.d, cdc}
@attribute MYCT real
@attribute MMIN real
@attribute MMAX real
@attribute CACH real
@attribute CHMIN real
@attribute CHMAX real
@attribute class real
@data
cdc,125,256,6000,256,16,128,199
amdahl,29,8000,32000,32,8,32,253
apollo,29,8000,32000,32,8,32,253
...
```

Numeric Prediction with M5P

```
=== Run information ===
Scheme:weka.classifiers.trees.M5P -M 4.0
Relation: cpu
Instances:209
Attributes:8
              vendor
              MYCT
              MMIN
              MMAX
              CACH
              CHMIN
              CHMAX
              class
Test mode:split 95.0% train, remainder test

=== Classifier model (full training set) ===
M5 pruned model tree:
(using smoothed linear models)

MMAX <= 14000 : LM1 (141/2.365%)
MMAX >  14000 : LM2 (68/28.342%)

LM num: 1
class =
        -2.0542 * vendor=honeywell,ipl,ibm,cdc,ncr,basf,gould,siemens,
        + 5.4303 * vendor=adviser,sperry,amdahl
        - 5.7791 * vendor=amdahl
        + 0.0064 * MYCT
        + 0.0016 * MMIN
        + 0.0034 * MMAX
        + 0.5524 * CACH
        + 1.1411 * CHMIN
        + 0.0945 * CHMAX
        + 4.1463
```

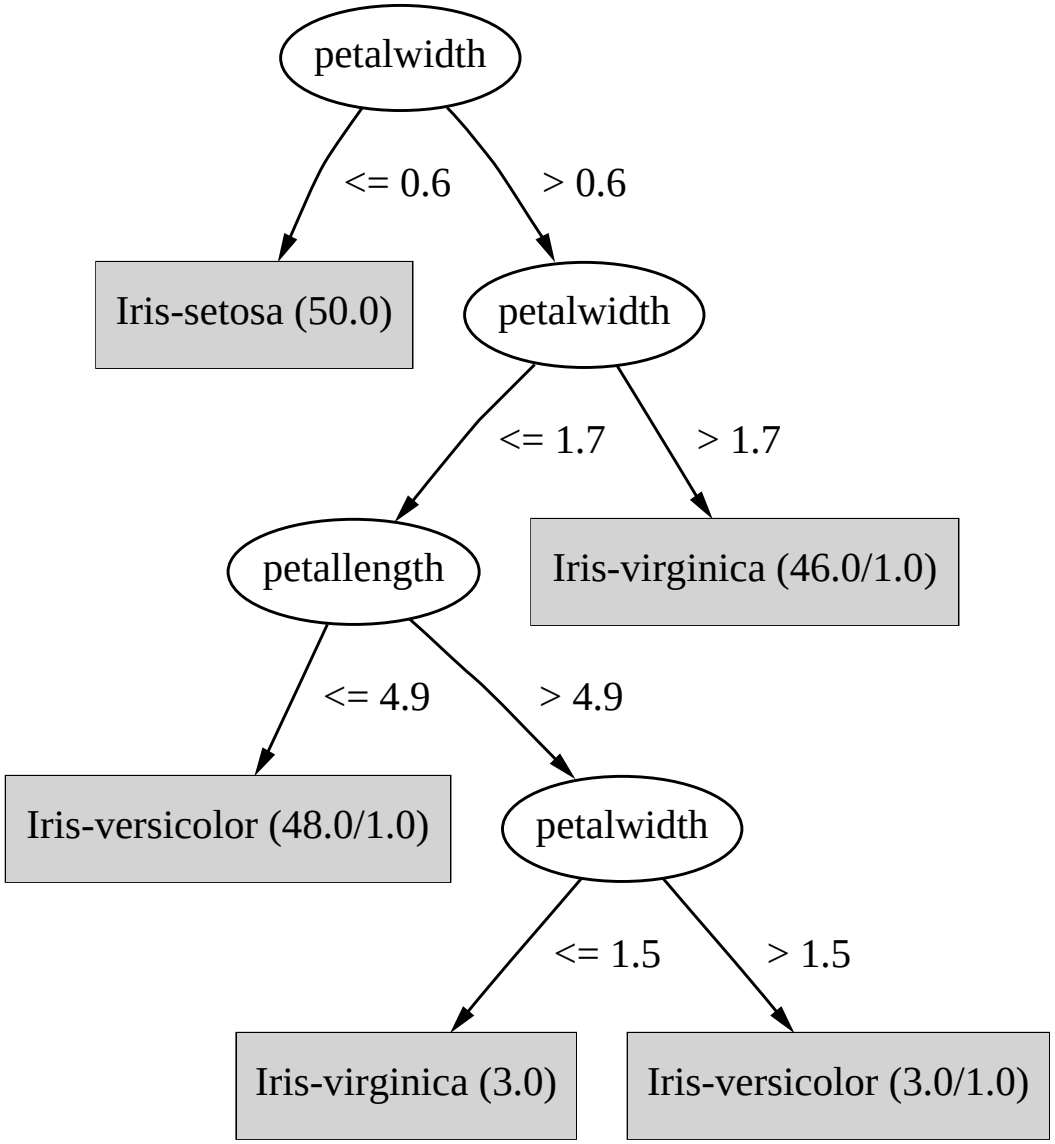
```
LM num: 2
class =
        -57.3649 * vendor=honeywell,ipl,ibm,cdc,ncr,basf,gould,siemens,
        + 46.1469 * vendor=adviser,sperry,amdahl
        - 58.0762 * vendor=amdahl
        + 0.012 * MYCT
        + 0.0162 * MMIN
        + 0.0086 * MMAX
        + 0.8332 * CACH
        - 1.2665 * CHMIN
        + 1.2741 * CHMAX
        - 107.243

Number of Rules : 2
Time taken to build model: 0.16seconds

=== Predictions ontest split===
inst#,    actual, predicted, error
     1    120        124.956      4.956
     2     27         25.865     -1.135
     3    102        100.884     -1.116
     4     42         44.458      2.458
     5     20         15.218     -4.782
     6     30         25.96      -4.04
     7    107         98.92      -8.08
     8    181        230.493     49.493
     9    603        573.598    -29.402
    10     21         12.618     -8.382

=== Evaluation on test split ===
=== Summary ===
Correlation coefficient                  0.9939
Mean absolute error                     11.3844
Root mean squared error                 18.7668
Relative absolute error                 11.7003 %
Root relative squared error             11.072  %
Total Number of Instances               10
```

Decision Tree Classifier for Iris Data



Decision Tree Classifier

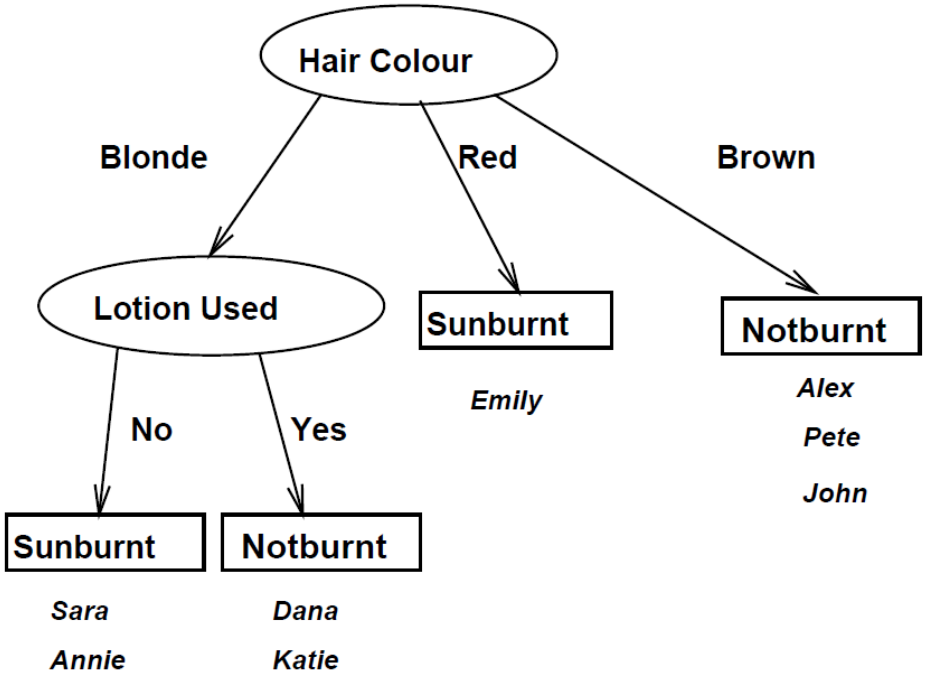
Suppose we have the following data. How can we use it to decide whether some unknown person will get sunburnt?

name	Hair	Height	Weight	Lotion	Result
Sara	blonde	average	light	no	sunburnt
Dana	blonde	tall	average	yes	not burnt
Alex	brown	short	average	yes	not burnt
Annie	blonde	short	average	no	sunburnt
Emily	red	average	heavy	no	sunburnt
Peter	brown	tall	heavy	no	not burnt
John	brown	average	heavy	no	not burnt
Katie	blonde	short	light	yes	not burnt

1. We could use (K) Nearest Neighbour, OneR, ZeroR
2. Can we do better?
3. Most popular, most versatile classifier, the Decision Tree
4. Ross Quinlan, Australia, 1980's, C4.5, then C4.8
5. Refined, implemented in Weka as J48
6. Winston identification tree = decision tree

Decision Tree Classifier

- A leaf indicates a class
- An internal node is a decision node with a test to be carried out and a sub tree for each outcome.

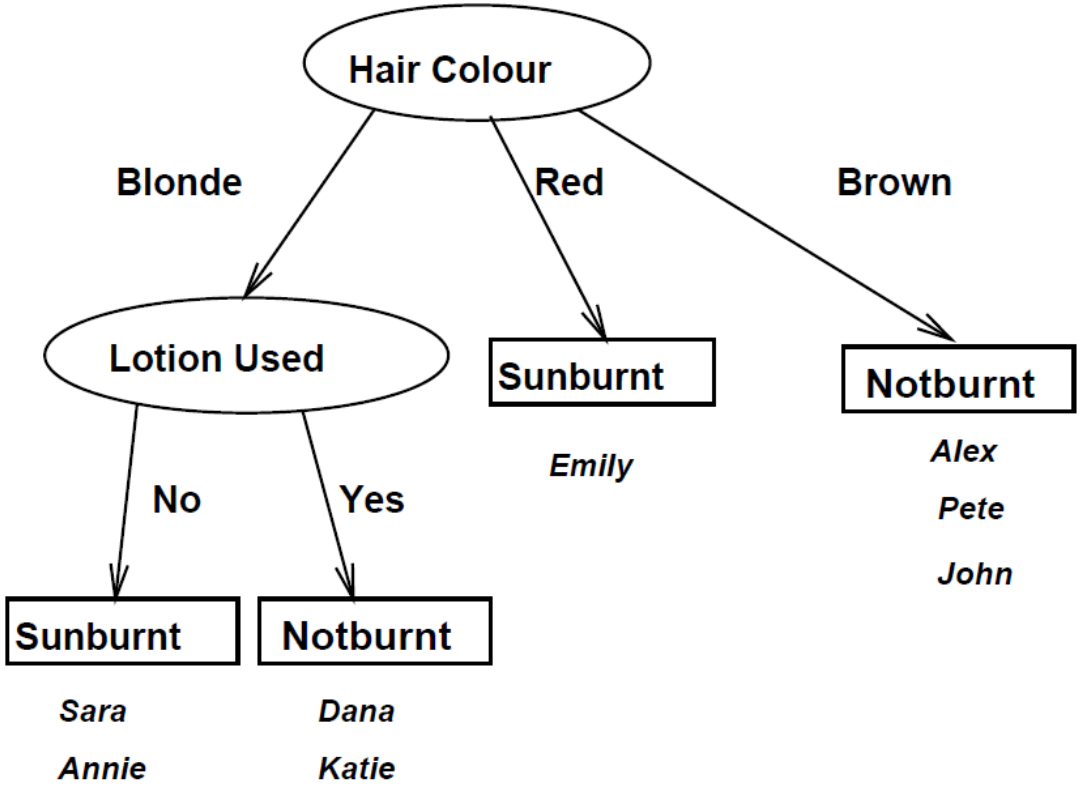


- Will Jack get sunburnt?

Name	Hair	Height	Weight	Lotion
Jack	blond	average	heavy	no

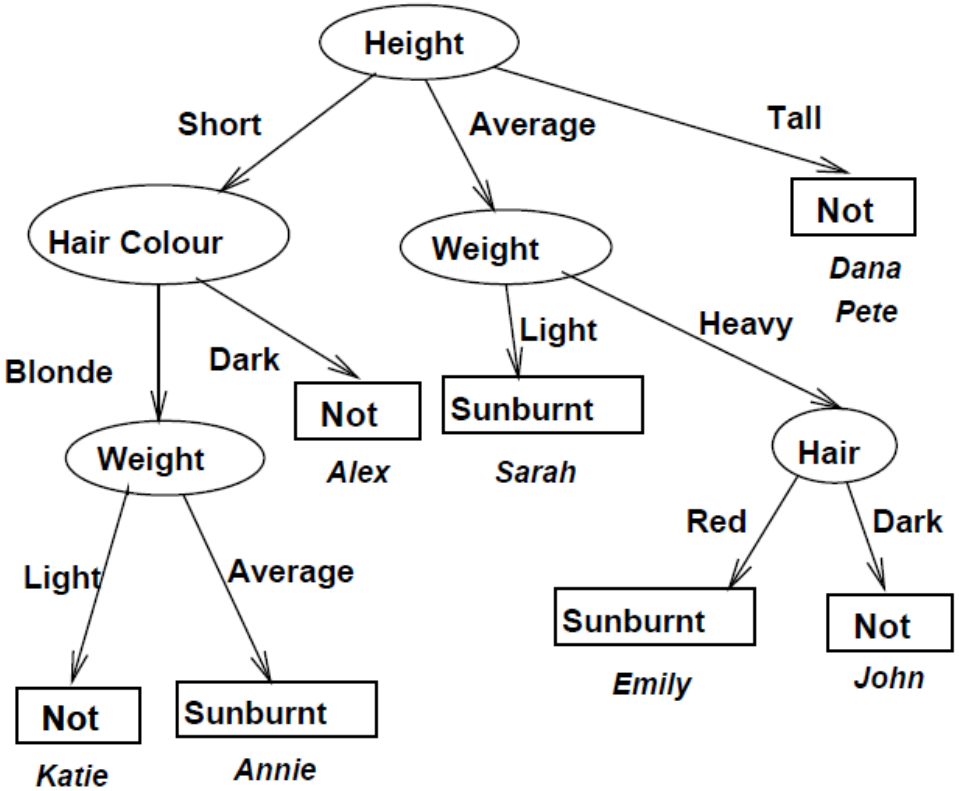
Start from root, carry out the test and follow indicated branch to a leaf.

Line Printer Form of Decision Tree




```
Hair Colour = Blonde:
|   Lotion Used = no: Sunburnt
|   Lotion Used = yes: Not burnt
Hair Colour = Red: Sunburnt
Hair Colour = Brown: Not Burnt
```

Alternative Decision Tree



- Does this tree give the same classifications as the previous one?
- What is its line printer form?
- Which tree is preferable? Why?
  - Tree 1 is smaller (Occam's Razor)
  - Knowledge in tree 1 is a much better fit to what is known in the domain.

Decision Tree Algorithm

```
Subroutine Create tree(Examples)
  IF      all examples belong to the same class
  THEN    Return a leaf node with the class as label
  ELSE    Attribute = Select an Attribute(Examples)
          Node = Create Test Node(Attribute)
          For each value of Attribute
             Partition = Subset of examples with attribute value
             Subtree = Create tree(Partition)
          Attach(Subtree,Node,Value)
          Return Node
Main program
Create tree(Training Set)
```

- The basic idea is to keep splitting the training set into smaller pieces until each piece has only members of one class.
- Which attribute to select for splitting?
- Choose attribute split which minimizes disorder.

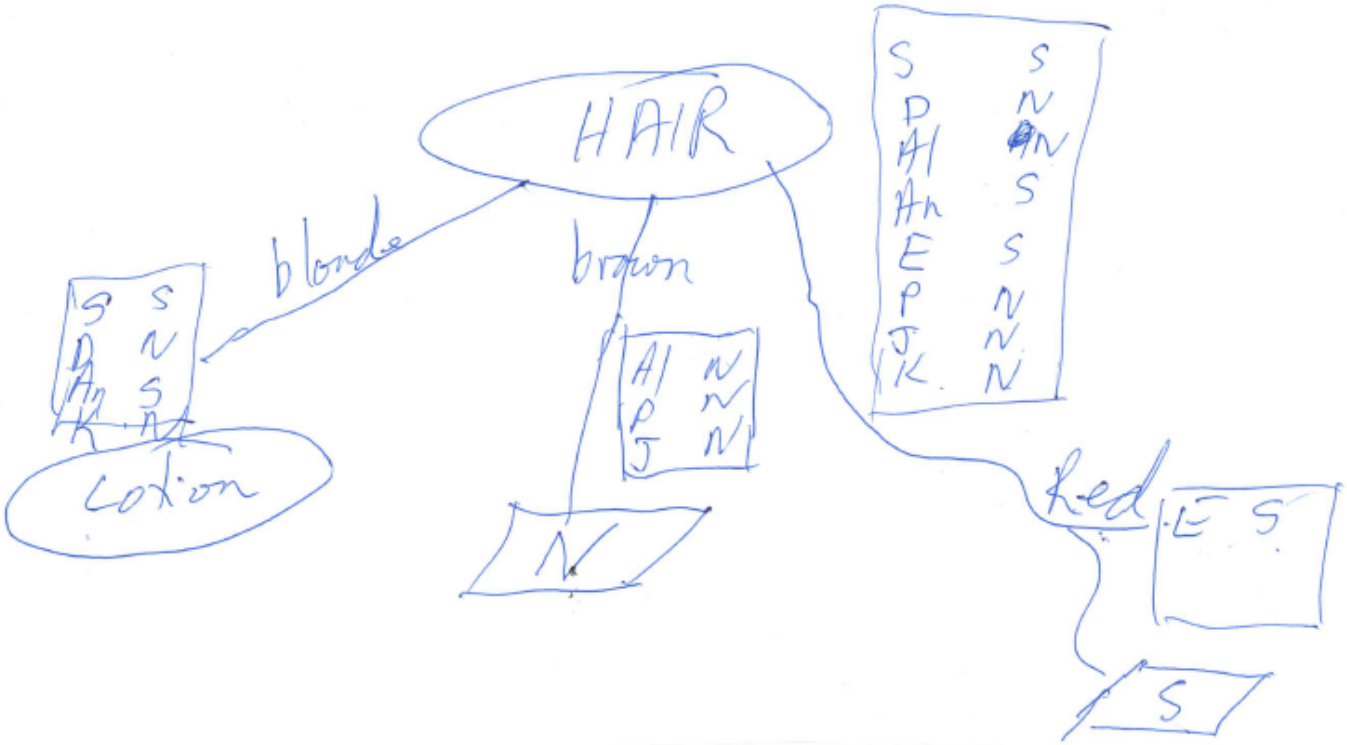
Decision Tree Classifier

Hair

S	S
D	N
Al	N
An	S
E	S
P	N
J	N
K	N

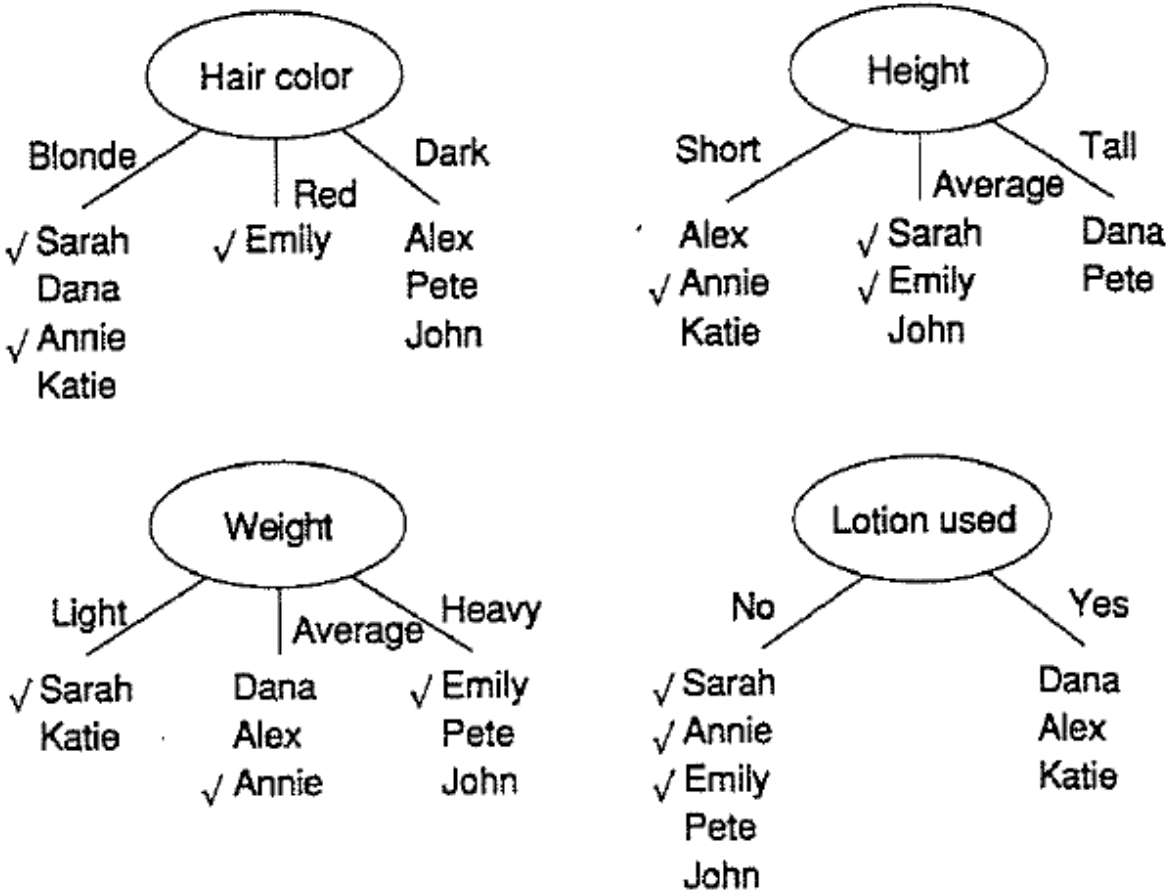
name	Hair	Height	Weight	Lotion	Result
Sara	blonde	average	light	no	sunburnt
Dana	blonde	tall	average	yes	not burnt
Alex	brown	short	average	yes	not burnt
Annie	blonde	short	average	no	sunburnt
Emily	red	average	heavy	no	sunburnt
Peter	brown	tall	heavy	no	not burnt
John	brown	average	heavy	no	not burnt
Katie	blonde	short	light	yes	not burnt

Decision Tree Classifier



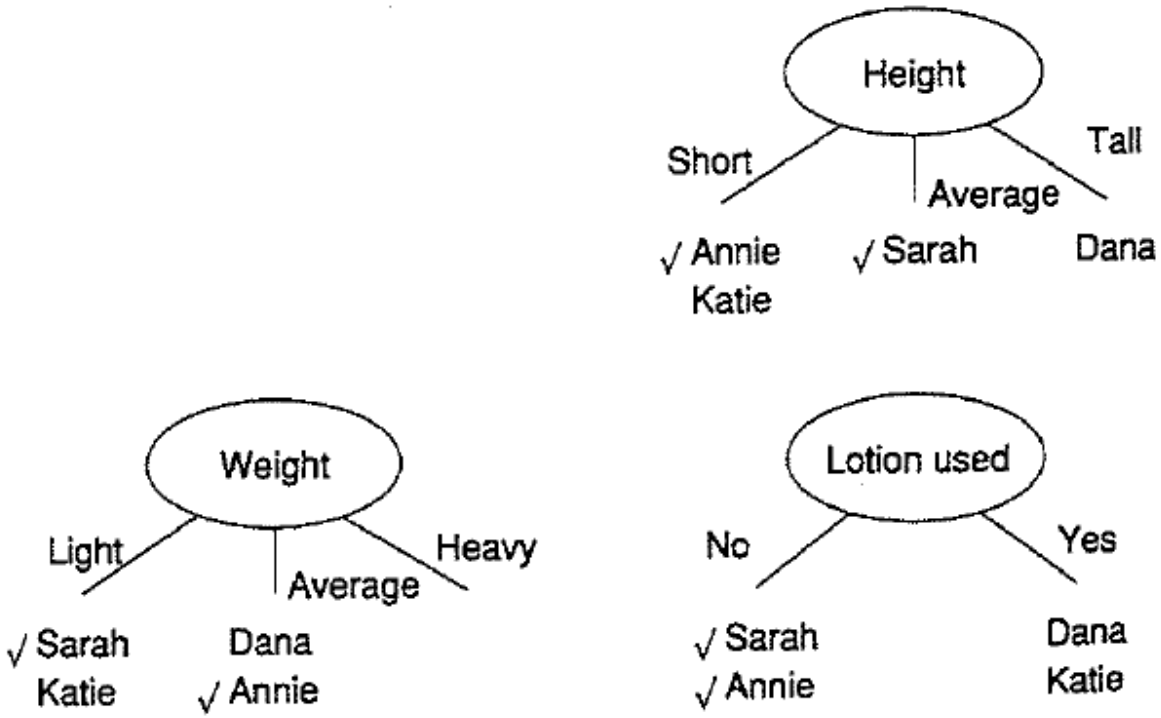
name	Hair	Height	Weight	Lotion	Result
Sara	blonde	average	light	no	sunburnt
Dana	blonde	tall	average	yes	not burnt
Alex	brown	short	average	yes	not burnt
Annie	blonde	short	average	no	sunburnt
Emily	red	average	heavy	no	sunburnt
Peter	brown	tall	heavy	no	not burnt
John	brown	average	heavy	no	not burnt
Katie	blonde	short	light	yes	not burnt

Decision Tree Classifier



Each test divides the sunburn dataset into different subsets. Ticked names are those got sunburns, and no ticks mean no sunburns. Intuition suggests that the test on hair-color attribute does the best job in dividing the dataset into homogeneous subsets.

Decision Tree Classifier



Once the blonde-haired people have been isolated, we continue to test on remaining attributes. The tests show that the attribute lotion does the best job of dividing the blonde-haired set consisting of Sarah, Dana, Annie, and Katie into homogeneous subsets.

Measure of Disorder

Use a result from information theory:

n_b number of examples in branch b

n_t total number of examples in all branches

n_{bc} total number of examples in branch b of class c

Disorder at the end of branch b is

$$\left(\sum_c -\frac{n_{bc}}{n_b} log_2 \frac{n_{bc}}{n_b}\right)$$

Disorder is low (close to 0) if almost all examples are of the same class. Disorder is high (close to 1) if almost all examples are of different classes.

Example if we have two classes, 1 and 2, with 50 examples each:

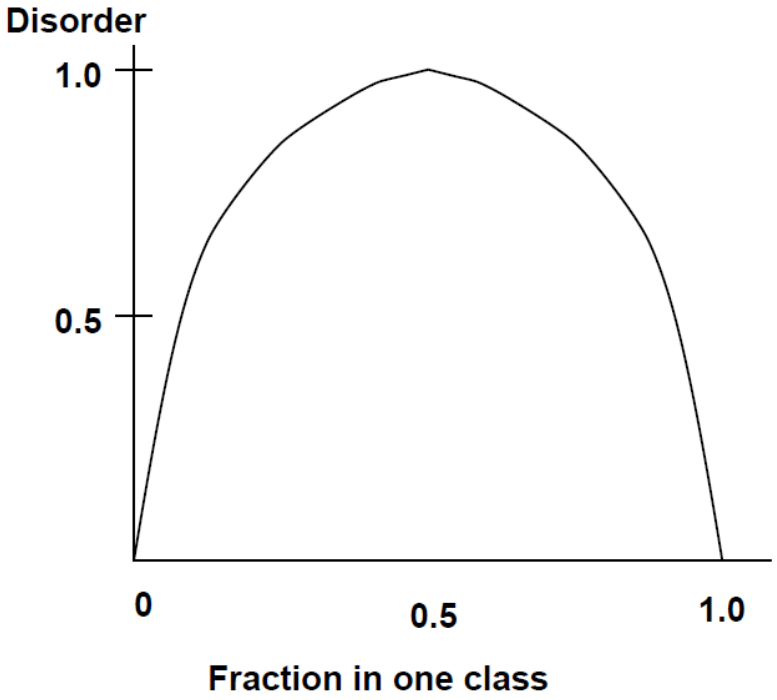
$$n_{b1} = 50 \qquad n_{b2} = 50 \qquad n_b = 100$$

$$disorder = -\frac{50}{100} log_2 \frac{50}{100} - \frac{50}{100} log_2 \frac{50}{100} = \frac{1}{2} + \frac{1}{2} = 1$$

$$(log_2 \frac{1}{2} = -1)$$

Measure of Disorder

Disorder vs fraction in one class for TWO classes



The notion of “disorder" is also referred to as “entropy” in Information Theory (founded by Claude Shannon), or “surprise", “uncertainty”, etc. Further information can be found here:

`https://en.wikipedia.org/wiki/Entropy_(information_theory)`

Choosing the Split Attribute

For a split on any attribute

$$Average\ disorder = \sum_b \left(\frac{n_b}{n_t}\right) \times (Disorder\ in\ branch\ b)$$

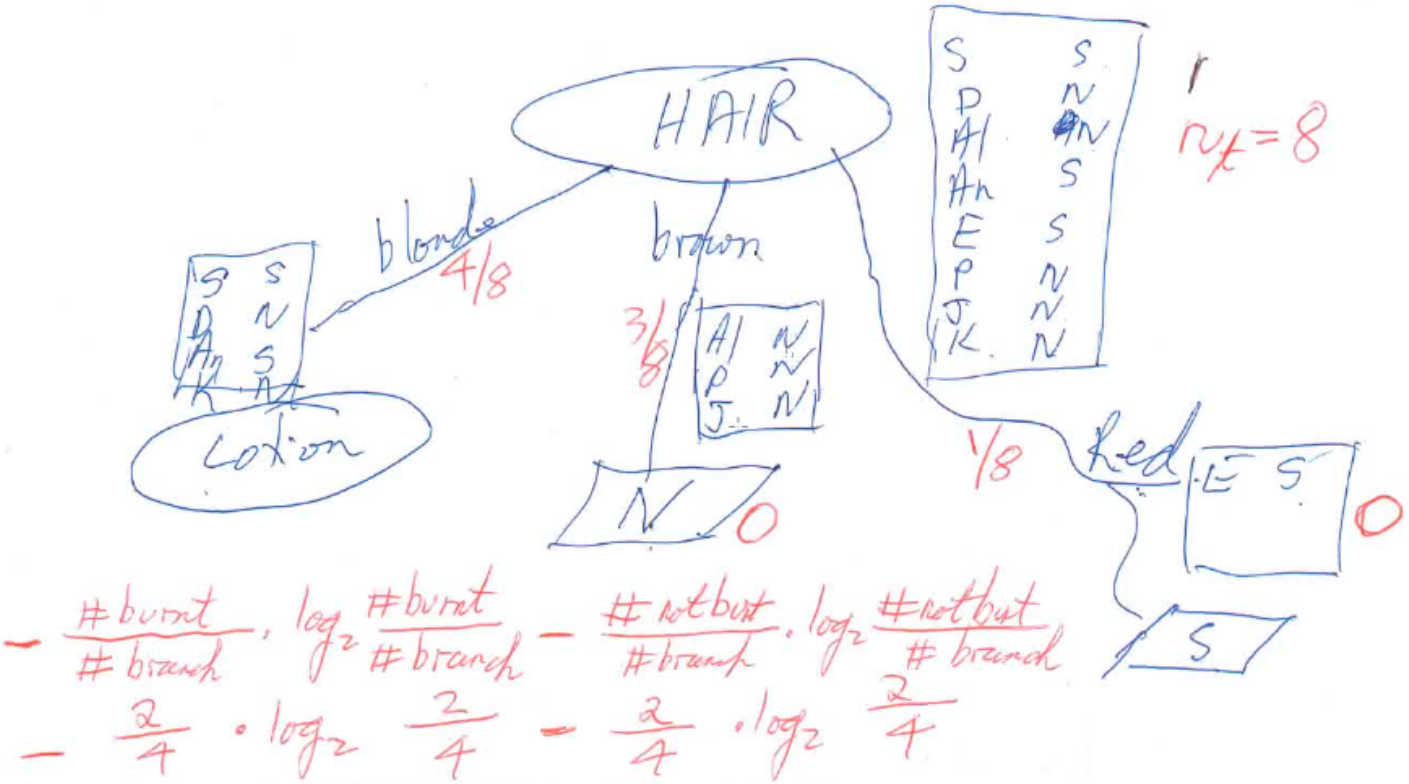
$$Average\ disorder = \sum_b \left(\frac{n_b}{n_t}\right) \times \left(\sum_c -\frac{n_{bc}}{n_b} log_2 \frac{n_{bc}}{n_b}\right)$$

Work out the disorder for each attribute.
Choose the attribute with LOWEST disorder.

Test	Disorder
Hair	0.50
Height	0.69
Weight	0.94
Lotion	0.61

Simple option, but not the best

Choosing the Split Attribute



$$- \frac{\#\text{burnt}}{\#\text{branch}} \cdot \log_2 \frac{\#\text{burnt}}{\#\text{branch}} - \frac{\#\text{not burnt}}{\#\text{branch}} \cdot \log_2 \frac{\#\text{not burnt}}{\#\text{branch}}$$

$$- \frac{2}{4} \cdot \log_2 \frac{2}{4} - \frac{2}{4} \cdot \log_2 \frac{2}{4}$$

$$Average\ disorder = \sum_b \left(\frac{n_b}{n_t}\right) \times (Disorder\ in\ branch\ b)$$

$$Average\ disorder = \sum_b \left(\frac{n_b}{n_t}\right) \times \left(\sum_c -\frac{n_{bc}}{n_b} log_2 \frac{n_{bc}}{n_b}\right)$$

Problems with Basic Algorithm

1. Overfitting – The decision tree works well on the data from which it was constructed but not on other data from the same population.
2. How can we get the most accurate tree? Examine all possible trees?
3. Which attribute to choose for splitting?
4. Should we split at all?

Splitting

How to choose the attribute for splitting?
disorder = entropy = information
Switch to C4.5/J48 terminology

1. Maximize information gain
2. Maximize gain ratio

- For 8 equally probable messages the information in any one is

$$-log_2(1/8) = 3\ bits$$

- A message that a case in a set S is in class C_j has probability

$$\frac{freq(C_j, S)}{|S|}$$

- information in the message is

$$-log_2\left(\frac{freq(C_j, S)}{|S|}\right)$$

Information Gain

- Information in a (training) data set T containing k classes is (Non equally probable messages need to be weighted by probability (frequency).)

$$info(T) = -\sum_{j=1}^{k} \frac{freq(C_j, T)}{|T|} \times log_2 \left(\frac{freq(C_j, T)}{|T|} \right) \quad bits$$

- Assume we have example set T and are making a decision to split.
- Current information is $info(T)$
- After splitting on attribute X with k values the information will be

$$info_X(T) = \sum_{i=1}^{k} \frac{|T_i|}{|T|} \times info(T_i)$$
$$gain(X) = info(T) - info_X(T)$$

- gain criterion – select attribute split to maximize information gain (mutual information); used by ID3

Gain Ratio

- Gain criterion is biased toward attributes with many values
  - If each example has a unique record number $info_X(T)$ will be 0 so $gain(X)$ will be maximum
  - This split is useless
- Put in a normalizing factor for this bias

$$split\ info(T) = -\sum_{i=1}^{n} \frac{|T_i|}{|T|} \times log_2\left(\frac{|T_i|}{|T|}\right)$$

- The 'split info' here considers only the number and size of the children nodes
- Then choose attribute split that maximizes this ratio

$$gain\ ratio = \frac{gain(X)}{split\ info(X)}$$

Gain Ratio - the sunburn data set

$$info(T) = -\frac{3}{8} \times log_2\frac{3}{8} - \frac{5}{8} \times log_2\frac{5}{8} = 0.9544$$

$$\begin{aligned} info_{Hair}(T) &= \frac{4}{8} \times (-\frac{2}{4} \times log_2\frac{2}{4} - \frac{2}{4} \times log_2\frac{2}{4}) \\ &+ \frac{3}{8}(-1 \times log_2 1) + \frac{1}{8}(-1 \times log_2 1) \\ &= 0.5 \end{aligned}$$

$$gain(Hair) = info(T) - info_{Hair}(T) = 0.9544 - 0.5 = 0.4544$$

$$split\ info(T) = -\frac{4}{8} \times log_2\frac{4}{8} - \frac{3}{8} \times log_2\frac{3}{8} - \frac{1}{8} \times log_2\frac{1}{8} = 1.4056$$

$$gain\ ratio(Hair) == \frac{0.4544}{1.4056} = 0.3233$$

Preventing Overfitting

- How to get most accurate tree (on WITHHELD data)?
- Prevent the tree from becoming too complex
- Pre-pruning or Stopping
  - Only split if the information gain will be significant. ID3 used (χ^2) to test for significance
  - Not as good in practice as post pruning
- Post pruning
  - Build complete tree first
  - Make the tree less complex and hopefully more general by:
    - replacing subtrees with leaves
    - replacing subtrees with most common branch

Pruning of Decision Trees

- If a branch is replaced by a leaf, the leaf will correspond to several classes. Its label will be the most common class.
- Error on training set will increase
- Outline of the approach
  1. Assume that one can (magically) predict error rate of (sub) tree and leaf
  2. Start from bottom, examine each non leaf subtree
  3. if predicted error would be lower if this sub tree was a leaf then replace with a leaf
  4. if predicted error would be lower if this sub tree replaced with most common branch, then replace

Predicting Error Rate of a Subtree

- Can't use error rate on training set directly
- Approach 1. Apply tree to a separate set of cases, validation set [Weka Reduced Error Pruning]
  - Break the original training set randomly into
  - Actual training set
  - Validation set
- Approach 2. Use some smart statistical theory
  1. We know that training set error will always be lower than test set error
  2. Build a statistical model of this difference
  3. Use the model to adjust (raise) the error on training set for pruning.
  4. C4.5/J48 Pruning confidence

Continuous Attributes

- Much more straight forward than it might appear
- Sort on values of attribute A, get $v_1, v_2 \ldots v_m$
- Can select threshold:

$$\frac{v_i + v_{i+1}}{2}$$

- $m - 1$ possible splits to examine
- C4.5 selects largest value of A that does not exceed midpoint
- Decision trees and rules have numbers actually in the data

Running J48

- Important Parameters
  - Minimum number of objects. Do not perform a split if the node has fewer than this number of examples.
  - Pruning Confidence. Smaller values cause more heavy pruning. Adjust if error rate on pruned trees for test cases is much higher than estimated error rate.
  - Reduced error pruning
- Output
  - Visualise the tree
  - Visualize the errors

Other Induction Programs

- There are dozens of other programs for generating decision trees from data based on different algorithms.
- There are also many programs for generating classification rules directly from data.
- Decision trees or rules are often preferred for data mining over nearest neighbour and neural network methods because they can:
  - Give insight into the data
  - Be used to explain how a decision (eg fraud/not fraud) has been made
  - What do you conclude if
    - If the accuracy of OneR and J48 are very similar, eg 94% vs 98%?
    - If the accuracy of OneR and J48 are far apart, eg 64% vs 90%?

Key Insight about Classification

1. Prepare the data, generate arff file
2. Choose classifier
   - Experiment with parameter settings
   - Want high test/cross-validation accuracy
   - Want no overfitting
3. Evaluate with error rate (or mean absolute error)

It is worth noting that:

- Highlighted in red are tasks that need to be done no matter what the classifier is.
- At step 2 you can find a good classifier without understanding all the details of all the parameters.
- After a classifier is chosen for deployment you need to get a good understanding of the algorithm and parameters.

Readings and Acknowledgment

- Readings:
  - Chapters 2 and 3 (background)
  - Sections 4.1, 4.3 (rules and decision trees)
  - Section 6.6 (regression tree, model tree)
  - Sections 5.1 and 5.3 (training and testing, and cross-validation)

  from the prescribed text:

  Ian H. Witten, Eibe Frank and Mark A Hall. Data Mining: Practical Machine Learning Tools and Techniques, Morgan Kaufmann Publishers, 3rd edition, 2011.

- Acknowledgment: These slides are based on those originally developed by Vic Ciesielski in the previous semesters.